Software Engineering Project

Requirement Specification & Design Document

Hotel Management System

Group 03

Nguyen Cong Thanh	(Leader)
Luong Nhat Khoi	(Deputy Leader)
Nguyen Hoang Bao Anh	(Member)
Nguyen Minh Man	(Member)
Huynh Nhat Hoa	(Member)

April 5, 2026

Contents

1 Introduction **2**

2 Requirement Specification **2**
2.1 Functional Requirements 2
2.1.1 Authentication & Authorization 2
2.1.2 Admin Capabilities 2
2.1.3 Customer Capabilities 2
2.2 Non-Functional Requirements 3

3 System Modeling **3**
3.1 Data Flow Diagram (DFD) 3
3.2 Use Case Diagram 4

4 System Design **5**
4.1 Architectural Design (Class Diagram) 6
4.2 Data Model (Database Schema) 7
4.2.1 Key Relationships and Constraints 8
4.3 Interface Design Description 8

5 Implementation & Testing Strategy **9**
5.1 Implementation Stack 9
5.2 Testing Approach 9

1 Introduction

This document serves as the comprehensive Requirement Specification and System Design for the Hotel Management System developed by Group 03. The project aims to deliver a robust, desktop-based application utilizing Python, Tkinter for the graphical user interface, and SQLite for relational data management.

2 Requirement Specification

2.1 Functional Requirements

The system functions are categorized based on user roles: **Admin** and **Customer**.

2.1.1 Authentication & Authorization

- **Login/Logout:** Users can securely log in to their designated portals based on their assigned roles.
- **Registration:** New customers can create accounts by providing personal information and setting up security questions.
- **Password Recovery:** Users can reset forgotten passwords by correctly answering their predefined security questions.

2.1.2 Admin Capabilities

- **Dashboard & Reporting:** View real-time daily activity timelines, occupancy metrics, and financial summaries.
- **Customer Management:** Perform CRUD (Create, Read, Update, Delete) operations on customer records, enforcing constraints against deleting customers with active bookings.
- **Booking Management:** Approve, reject, check-in, or check-out bookings. Admins can directly reserve rooms for customers ensuring no date-range overlapping occurs.
- **Room Management:** Filter and view room statuses (Available, Occupied, Reserved).

2.1.3 Customer Capabilities

- **View Inventory:** Browse available rooms and their respective rates.
- **Booking Requests:** Submit booking requests for specific dates. The system validates availability before submitting the request as 'Pending'.
- **Booking History:** Track the status of personal booking requests.

2.2 Non-Functional Requirements

- **Security:** Passwords and security answers must be hashed and salted using the PBKDF2-HMAC (SHA256) algorithm with 120,000 iterations.
- **Data Integrity:** The SQLite database must enforce Foreign Key constraints to prevent orphaned records. Strict date-range validation prevents room double-booking.
- **User Interface (UI):** The system features a "Luxury Classic" design system characterized by a Navy, Gold, and Ivory color palette, ensuring a premium user experience without requiring full page reloads (module-based routing).
- **Maintainability:** Code is structured into logical modules (`hotel.py`, `database.py`, `views.py`, `ui_logic.py`) ensuring separation of concerns.

3 System Modeling

3.1 Data Flow Diagram (DFD)

The DFD illustrates the flow of information between external entities and the system's internal processes and data stores.

- **Level 0 (Context Diagram):** Depicts the system as a single process interacting with *Admin* and *Customer* entities. Customers input credentials and booking requests, receiving availability and history. Admins input operational commands and receive analytics and system states.

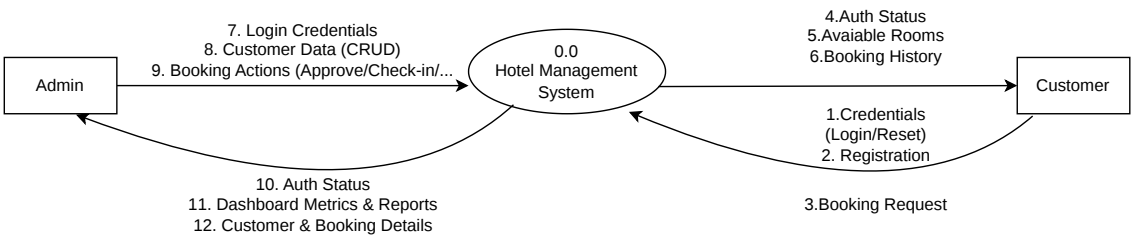

Figure 1: Data Flow Diagram - Level 0

- **Level 1 (Detailed Process):** Decomposes the system into four main processes: *1.0 Authentication*, *2.0 Customer Management*, *3.0 Booking Management*, and *4.0 Dashboard & Reporting*. Data stores include *D1: Users*, *D2: Customers*, *D3: Rooms*, and *D4: Bookings*.

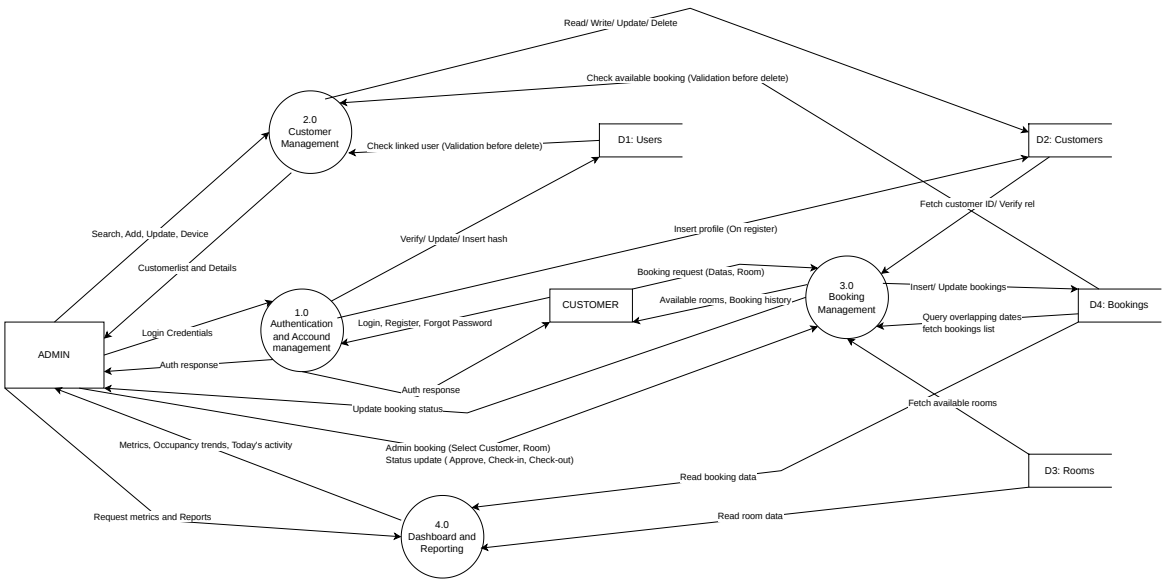

Figure 2: Data Flow Diagram - Level 1

3.2 Use Case Diagram

The Use Case diagram maps the interactions between Actors and System Functions.

- **Actors:** Admin, Customer.
- **Key Relationships:**
  - `«include»`: Creating an Admin Booking explicitly requires Checking Room Availability.
  - `«extend»`: Forgot Password extends the Login process as an exceptional flow. Actions like Approve, Reject, Check-in extend the general Manage Bookings use case.
  - **Generalization:** Add, Update, and Delete Customer inherit from the abstract Manage Customers use case.

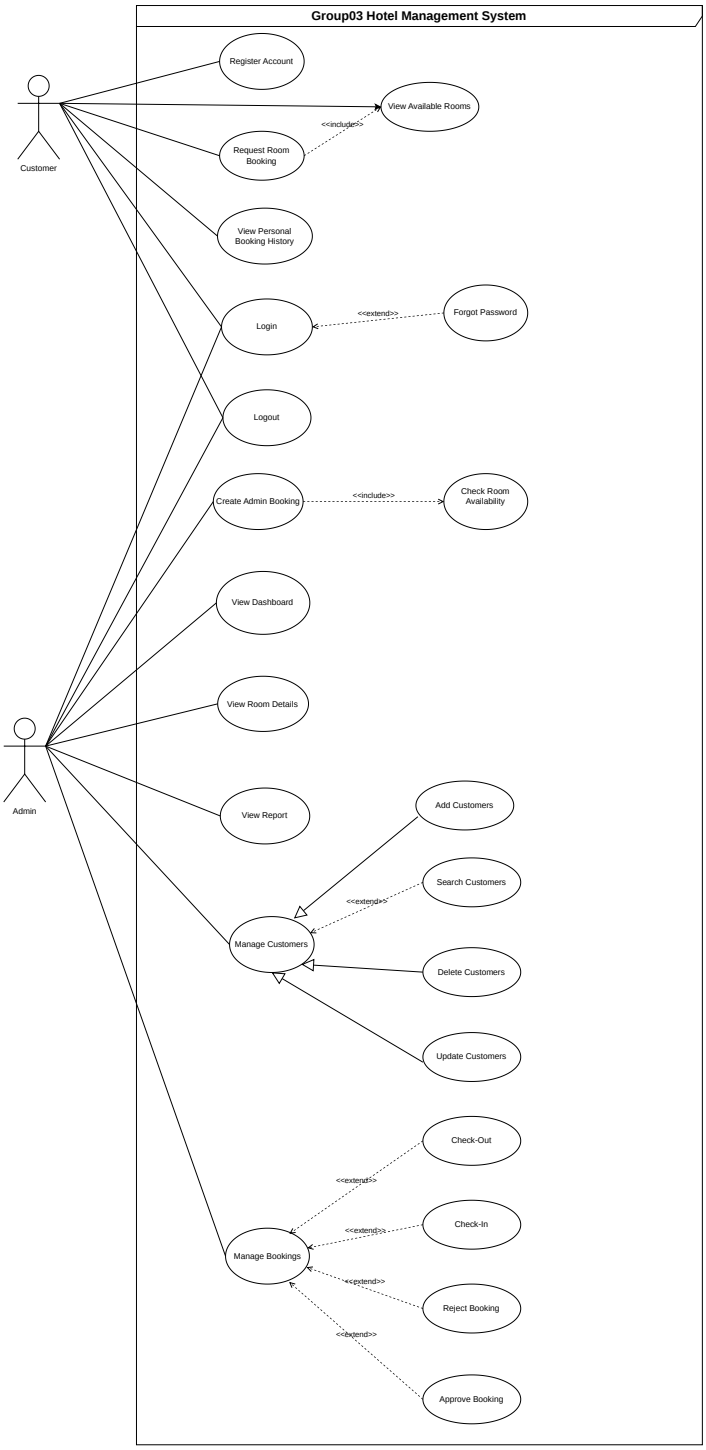

Figure 3: System Use Case Diagram

4 System Design

4.1 Architectural Design (Class Diagram)

The application follows a modular architecture that separates concerns between user interface, business logic, and data persistence, as implemented in `hotel.py` and supporting modules:

- **Controller (HotelManagementSystem):** The primary class that maintains the application's state, handles routing between views, and manages user session data (role, username, and customer ID).
- **View Layer (views.py):** A collection of rendering functions that construct the GUI components. It uses dependency injection by receiving the `app` instance to trigger controller-level callbacks.
- **Data Layer (database.py):** Encapsulates all SQL operations, providing a clean API for the controller to perform CRUD operations without writing raw SQL in the UI logic.
- **Logic Layer (ui_logic.py):** Contains utility functions for data processing, such as calculating dashboard metrics and formatting dates for display.

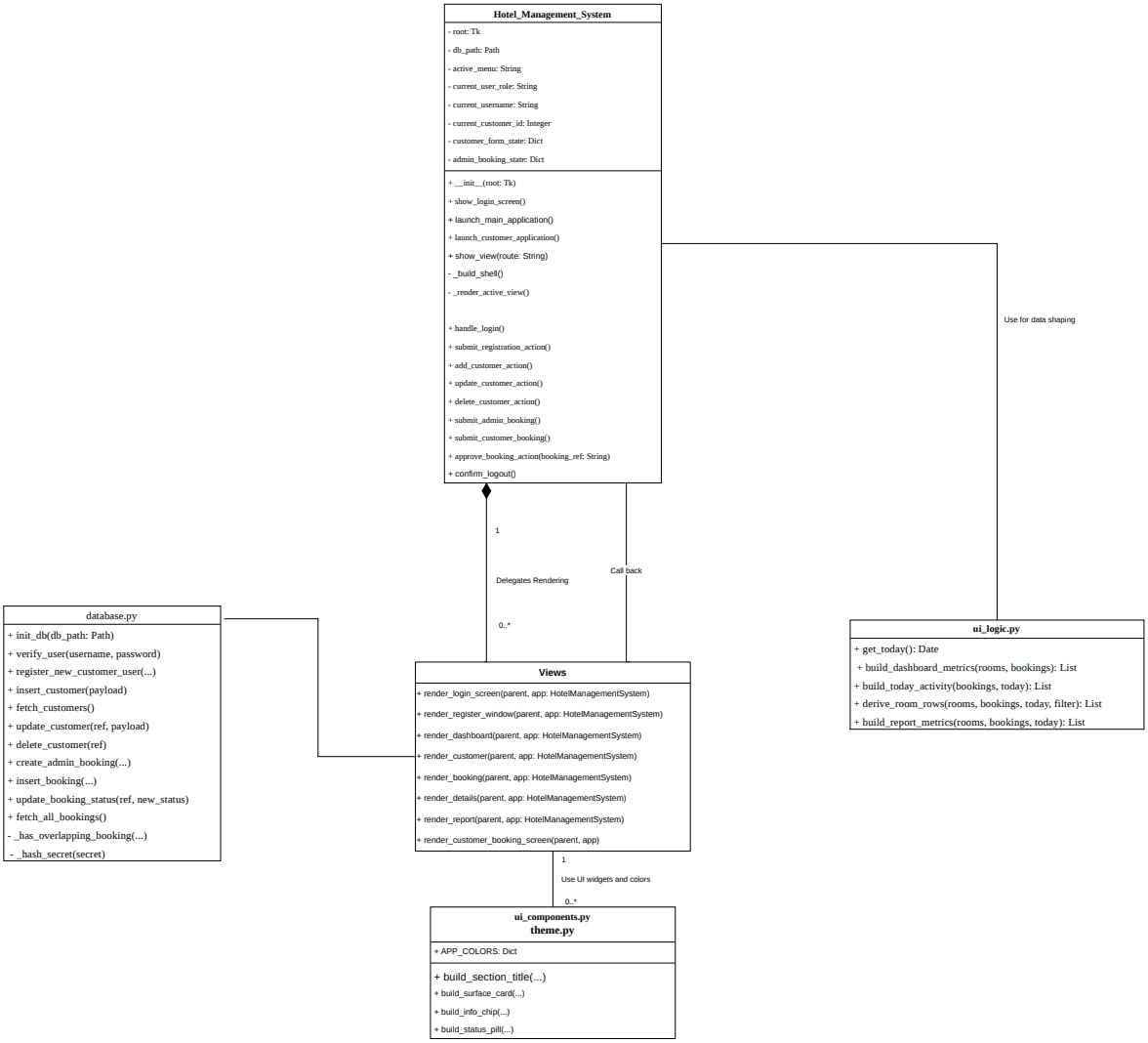

Figure 4: System Class and Module Interaction Diagram

4.2 Data Model (Database Schema)

The system utilizes a relational database (SQLite) with a normalized schema to ensure data consistency and integrity. The data model consists of four core entities as defined in the `database.py` module:

- **Customers:** Stores profile information including a unique reference code, name, contact details, and identity proofs.
- **Users:** Manages authentication data. It maintains a foreign key to the *Customers* table for registered clients, implementing a "Set Null" on delete constraint to preserve account history.
- **Rooms:** Defines the hotel's physical inventory, including room numbers, categories (Deluxe, Suite, etc.), and nightly rates.

- **Bookings:** The central transactional table that links customers to rooms. It stores stay dates, guest counts, and tracks the lifecycle of a reservation through statuses: *Pending, Reserved, Checked-in, Checked-out, or Cancelled.*

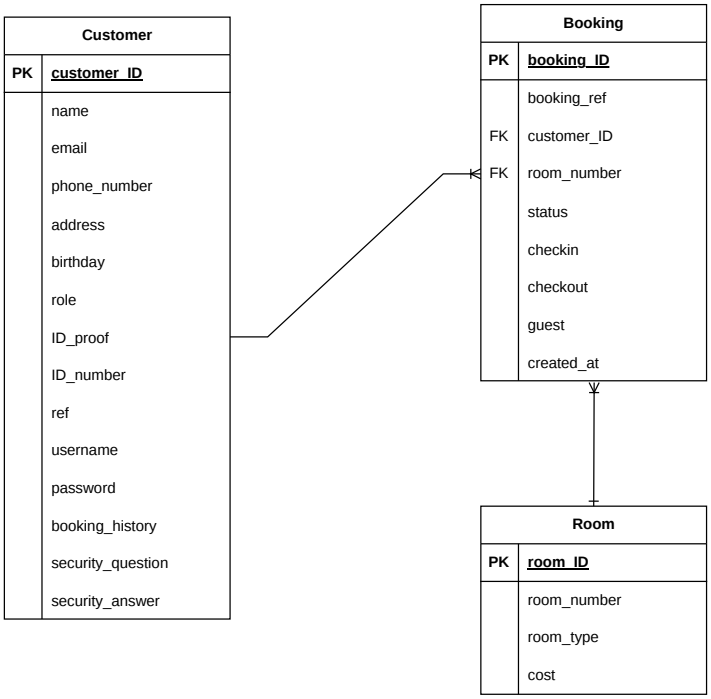

Figure 5: Entity Relationship Diagram (ERD)

4.2.1 Key Relationships and Constraints

- **Referential Integrity:** Foreign keys connect *Bookings* to both *Customers* and *Rooms* to ensure every reservation is tied to valid entities.
- **Unique Constraints:** Fields such as `Username`, `ref`, `room_number`, and `booking_ref` are marked as UNIQUE to prevent data duplication.
- **Business Logic Validation:** The database layer implements a strict overlap check to ensure no room can be booked for overlapping date ranges.

4.3 Interface Design Description

The UI uses Tkinter wrapped in custom styles to emulate modern web dashboards:

- **Admin Shell:** A three-pane layout featuring a persistent header, a left-side navigation sidebar, and a dynamic main content canvas with vertical scrolling.
- **Customer Portal:** A simplified dual-section view focusing on an immediate booking form juxtaposed with personalized booking history and available inventory.
- **Component System:** Reusable *Surface Cards*, *Info Chips*, and *Status Pills* ensure visual consistency across all modules.

5 Implementation & Testing Strategy

5.1 Implementation Stack

- **Language:** Python 3.x
- **GUI Framework:** Tkinter (Standard Library)
- **Database:** SQLite3 with `row_factory` for dict-like access.
- **Version Control:** Git / GitHub for collaborative development.

5.2 Testing Approach

- **Unit Testing:** Validating pure functions in `ui_logic.py` (e.g., date range overlap logic).
- **Integration Testing:** Ensuring `database.py` accurately executes transactions and rolls back on constraint violations (e.g., booking the same room twice).
- **Manual System Testing:** Using Excel test matrices to verify user flows, specifically the state transitions of a booking (Pending $\rightarrow$ Reserved $\rightarrow$ Checked-in $\rightarrow$ Checked-out).